

Production

Take your work off your laptop and into the world: trace every step with **LangSmith**, stream for responsive interfaces, and serve a chain or agent behind a **FastAPI** endpoint — with both **Ollama** and the **Gemini API**.

THIS VOLUME COVERS

Phase 5 — Production

Project: a deployed, traced API

The last mile

You can now build chains, RAG pipelines, and agents. This final part is about the difference between something that works on your machine and something other people can rely on. Three skills carry you across that line: **seeing** what your application actually did (observability), **delivering** answers smoothly (streaming), and **exposing** your work so other software can call it (serving).

None of these require rewriting what you have built. A chain or agent from the previous parts drops straight into everything here.

THE ENGINE TOGGLE, UNCHANGED TO THE END

```
PYTHON

from dotenv import load_dotenv
from langchain.chat_models import init_chat_model

load_dotenv()

llm = init_chat_model("ollama:llama3.1", temperature=0.3) # local
# llm = init_chat_model("google_genai:gemini-2.5-flash", temperature=0.3) # cloud
```

PHASE 5

Production

We wire in LangSmith tracing (a single environment variable), revisit streaming as the backbone of real-time UIs, serve a chain through a FastAPI endpoint with both a normal and a streaming route, and finish with a production checklist covering reliability, cost, and security. The capstone is a deployed, fully traced API.

5.1 LangSmith: seeing inside your app

LLM applications are non-deterministic and multi-step, which makes them hard to debug with print statements alone. When an agent gives a wrong answer, you need to know: which tool did it call? what exact prompt reached the model? how long did each step take, and how many tokens did it cost? **LangSmith** is LangChain's observability platform, and it answers all of these by recording a **trace** — a tree of every step in a run.

The best part: if you build with LangChain or LangGraph, you enable it with environment variables and **no code changes**. Sign up at smith.langchain.com, create an API key, and add to your `.env`:

```
.ENV

LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
LANGSMITH_PROJECT=langchain-course      # optional; groups your traces
```

That is the whole setup. Now any chain or agent you `invoke` or `stream` — on either engine — is captured automatically. Open the project in the LangSmith UI and you will see each run as an expandable tree: the outer chain, every nested model call with its full prompt and response, every tool call and result, plus latency and token counts at each node. This is the same step-by-step trace you printed by hand in Part 4, now recorded for every request.

To trace your own helper functions (not just LangChain objects), wrap them with `@traceable`:

```
PYTHON

from langsmith import traceable

@traceable
def clean_question(text: str) -> str:
    """Tidy user input before it enters the chain."""
    return text.strip().rstrip("?") + "?"
```

★ This is your debugger for non-determinism

When a user reports “the bot gave a weird answer,” you open the trace for that exact run and see precisely what the model saw and produced. Trying to reproduce LLM bugs without tracing is the single most painful part of shipping — turning LangSmith on early saves hours later.

5.2 Streaming for responsive interfaces

You met `stream()` back in Part 1. In production it is what makes an interface feel alive: instead of a user staring at a spinner for ten seconds, words appear as the model produces them. For web servers you use the *async* variants — `ainvoke` and `astream` — because a server handles many requests concurrently and should not block while waiting on a model.

```
PYTHON

import asyncio

async def demo():
    # astream yields the same chunks as stream, but without blocking the server
    async for chunk in chain.astream({"question": "Explain async in one line."}):
        print(chunk, end="", flush=True)
    print()

asyncio.run(demo())
```

Every Runnable you have built — a bare model, a piped chain, a RAG chain, an agent — supports `astream`. That uniformity is what lets the FastAPI endpoint on the next page stream *any* of your chains by changing a single line.

i Need to stream tool steps too?

For agents, plain `astream` streams the final answer. To stream intermediate progress (which tool is running, partial results), use `astream_events` or the agent's `stream_mode` options from Part 4. Start simple with token streaming; reach for events only when the UI needs to show the agent's work as it happens.

5.3 Serving a chain with FastAPI

To let a website, mobile app, or another service use your chain, you wrap it in a web API. **FastAPI** is the standard modern choice in Python: fast, async-native, and almost no boilerplate. We expose two routes — one that returns the whole answer as JSON, and one that streams tokens.

● ● ● TERMINAL

```
pip install -U fastapi "uvicorn[standard]"
```

● ● ● PYTHON (APP.PY)

```
# app.py
from dotenv import load_dotenv
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()  # also loads LANGSMITH_* -> tracing is on automatically

llm = init_chat_model("ollama:llama3.1", temperature=0.3)
# llm = init_chat_model("google_genai:gemin-2.5-flash", temperature=0.3)

chain = (
    ChatPromptTemplate.from_messages([
        ("system", "You are a concise, helpful assistant."),
        ("human", "{question}"),
    ])
    | llm
    | StrOutputParser()
)

app = FastAPI(title="LangChain Service")

class Query(BaseModel):  # validates the incoming JSON body
    question: str

@app.post("/chat")  # returns the full answer as JSON
async def chat(q: Query):
    answer = await chain.ainvoke({"question": q.question})
    return {"answer": answer}

@app.post("/chat/stream")  # streams tokens as they are produced
async def chat_stream(q: Query):
    async def tokens():
        async for chunk in chain.astream({"question": q.question}):
            yield chunk
    return StreamingResponse(tokens(), media_type="text/plain")
```

Start the server, then call it from another terminal:

● ● ● TERMINAL

```
uvicorn app:app --reload --port 8000
```

● ● ● TERMINAL

```
# Full answer as JSON
curl -X POST localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"question": "Explain RAG in one sentence."}'

# Streamed answer (-N disables curl buffering so you see it arrive live)
curl -N -X POST localhost:8000/chat/stream \
  -H "Content-Type: application/json" \
  -d '{"question": "Write a haiku about Python."}'
```

Swap in any chain you built

To serve the “chat with a PDF” RAG chain (Part 3) or the research agent (Part 4), replace the `chain` object and adjust the input shape. The route code barely changes — because every chain and agent is a Runnable with the same `ainvoke` / `astream` interface. Two other paths exist: **LangServe** (`add_routes`) auto-generates routes from a Runnable, and **LangGraph Platform** hosts agents with one command. Plain FastAPI is the most portable and is all most projects need.

5.4 Reliability, cost, and security

A demo tolerates failure; a service must not. Two Runnable methods harden any chain with one line each.

`with_retry` automatically re-attempts a step that hits a transient error, and `with_fallbacks` switches to a backup model if the primary fails — for example, falling back from cloud Gemini to local Ollama if the API is down.

```
PYTHON

# Retry transient failures (rate limits, network blips)
robust = chain.with_retry(stop_after_attempt=3)

# Fall back to a second model if the first errors out
from langchain.chat_models import init_chat_model
primary = init_chat_model("google_genai:gemin-2.5-flash")
backup = init_chat_model("ollama:llama3.1")
resilient_llm = primary.with_fallbacks([backup])
```

PRODUCTION CHECKLIST

Secrets

Keep keys in environment variables, never in code. Never return a key in an API response or log.

Observability

Turn on LangSmith from day one so production issues are traceable.

Reliability

Add `with_retry` on flaky steps and `with_fallbacks` for a backup model.

Cost

Pick the cheapest model that passes your quality bar (e.g. a Flash tier over a Pro tier), or run Ollama locally for zero per-token cost.

Latency

Stream to the user, and prefer smaller/faster models for interactive paths.

Input limits

Validate and cap request sizes (FastAPI + Pydantic do most of this) to control cost and abuse.

Safety

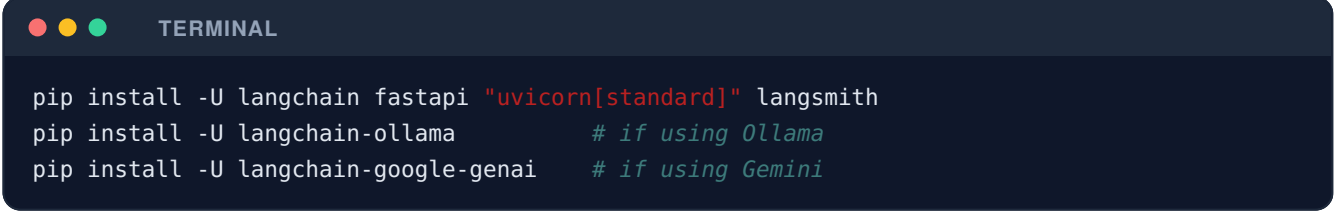
Only load FAISS indexes you created (`allow_dangerous_deserialization`), and sanitise anything you put into prompts.

i Cost is a design decision

The same chain can cost wildly different amounts depending on the model behind it. A common pattern: develop and test against free local Ollama, then point the engine toggle at a cloud model for production — and use LangSmith's token counts to confirm the bill before you scale.

Deploy a traced, streaming API

The capstone brings the whole series together: a deployable FastAPI service that streams, is observable through LangSmith, and is hardened with retries — serving a chain you can swap for any built earlier. Add the LangSmith variables to your `.env`, then run the file below.

A terminal window with a dark background and a title bar containing three colored circles (red, yellow, green) and the word "TERMINAL". The terminal displays three pip install commands. The first command is `pip install -U langchain fastapi "uvicorn[standard]" langsmith`. The second command is `pip install -U langchain-ollama` followed by a comment `# if using Ollama`. The third command is `pip install -U langchain-google-genai` followed by a comment `# if using Gemini`.

```
pip install -U langchain fastapi "uvicorn[standard]" langsmith
pip install -U langchain-ollama      # if using Ollama
pip install -U langchain-google-genai # if using Gemini
```

```

PYTHON (SERVICE.PY)

# service.py -- a production-shaped LangChain endpoint
from dotenv import load_dotenv
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()  # loads model keys AND LANGSMITH_TRACING / LANGSMITH_API_KEY

# --- choose your engine -----
llm = init_chat_model("ollama:llama3.1", temperature=0.3)
# llm = init_chat_model("google_genai:gemin-2.5-flash", temperature=0.3)
# -----

base_chain = (
    ChatPromptTemplate.from_messages([
        ("system", "You are a concise, helpful assistant."),
        ("human", "{question}"),
    ])
    | llm
    | StrOutputParser()
)
chain = base_chain.with_retry(stop_after_attempt=3)  # reliability

app = FastAPI(title="LangChain Service", version="1.0")

class Query(BaseModel):
    question: str = Field(min_length=1, max_length=2000)

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/chat")
async def chat(q: Query):
    return {"answer": await chain.ainvoke({"question": q.question})}

@app.post("/chat/stream")
async def chat_stream(q: Query):
    async def tokens():
        async for chunk in chain.astream({"question": q.question}):
            yield chunk
    return StreamingResponse(tokens(), media_type="text/plain")

```

Launch it and verify all three routes:



TERMINAL

```
uvicorn service:app --reload --port 8000
```



OUTPUT

```
$ curl localhost:8000/health
{"status":"ok"}

$ curl -X POST localhost:8000/chat -H "Content-Type: application/json" \
  -d '{"question":"What is LangChain in one sentence?"}'
{"answer":"LangChain is a framework for building applications powered by
large language models, with a standard interface across providers."}

# Open smith.langchain.com -> your project: every call above now appears
# as a full trace with prompt, response, latency, and token counts.
```



You shipped it

Point the engine toggle at either Ollama or Gemini, drop in the RAG chain or research agent in place of `base_chain`, and you have a real, observable, streaming LLM service. Deploy the same file to any host that runs a Python web app (a container, a VM, a serverless platform) and it behaves identically.

The series, complete

Five parts ago you had an empty terminal. You now have the full toolkit — and five projects that prove it.

What you built, phase by phase

0–1	Core First model call, invoke/stream, prompt templates	CALCULATOR CHATBOT
2	Chains & LCEL The pipe, parallelism, memory	MULTI-TURN ASSISTANT
3	RAG Embeddings, chunking, FAISS, retrieval	CHAT WITH A PDF
4	Agents @tool, create_agent, web search	RESEARCH AGENT
5	Production LangSmith, streaming, FastAPI	DEPLOYED, TRACED API

WHERE TO GO FROM HERE

- **LangGraph in depth.** When the prebuilt agent loop is not enough — multi-agent systems, branching workflows, human-in-the-loop approval — learn to build graphs directly. You already use it under the hood.
- **Evaluation with LangSmith.** Beyond tracing, LangSmith runs datasets and scores your app's answers, so you can measure whether a prompt or model change actually helped.
- **Persistence and scale.** Swap `InMemorySaver` for a database-backed checkpointer, move FAISS to a managed vector store, and put your FastAPI service behind proper auth and rate limiting.

* One framework, two engines, the whole journey

Every concept in this series worked identically on local **Ollama** and cloud **Gemini**, because that is the promise LangChain exists to keep: write your logic once, choose your engine freely. Keep that toggle handy — develop locally for free, ship on the cloud for quality, and never rewrite your application to do it.

End of Part 5 · The LangChain Practitioner's Guide · Series Complete